THE PIXEL ART MUSEUM INCIDENT

A SQL Forensic Case File

A digital masterpiece has vanished. The logs are encrypted. The clock is ticking. You are the lead database investigator, and your weapon is code.

LEAD INVESTIGATOR: DATABASE ADMIN // **STATUS:** ACTIVE INVESTIGATION

PRIORITY: *URGENT*

Incident Report INC-99

incident_id	description	time_of_event
INC-99	Digital Masterpiece "Pixel Mona Lisa" deleted from server.	2026-02-12 22:15:00

On Feb 12, 2026, at exactly 10:15 PM, the asset was wiped.

We have the what and the when. Now we need to find the who.

EVIDENCE

THE EVIDENCE LOCKER: DATABASE SCHEMA

To solve this, we must connect the dots. The `badge_id` connects people to actions. The `area_id` connects locations to logs.

Phase 1: Mapping the Crime Scene

The report states the art was deleted from the ‘Server Room.’ However, the security logs only list cryptic Area IDs (A1, A2, etc.).

We need to translate the physical room name into its corresponding code.

Terminal Window

```
> SELECT * FROM gallery_access;
```

QUERY: Retrieve all rows to find the ID for the Server Room.

EVIDENCE

Findings: The Location Key

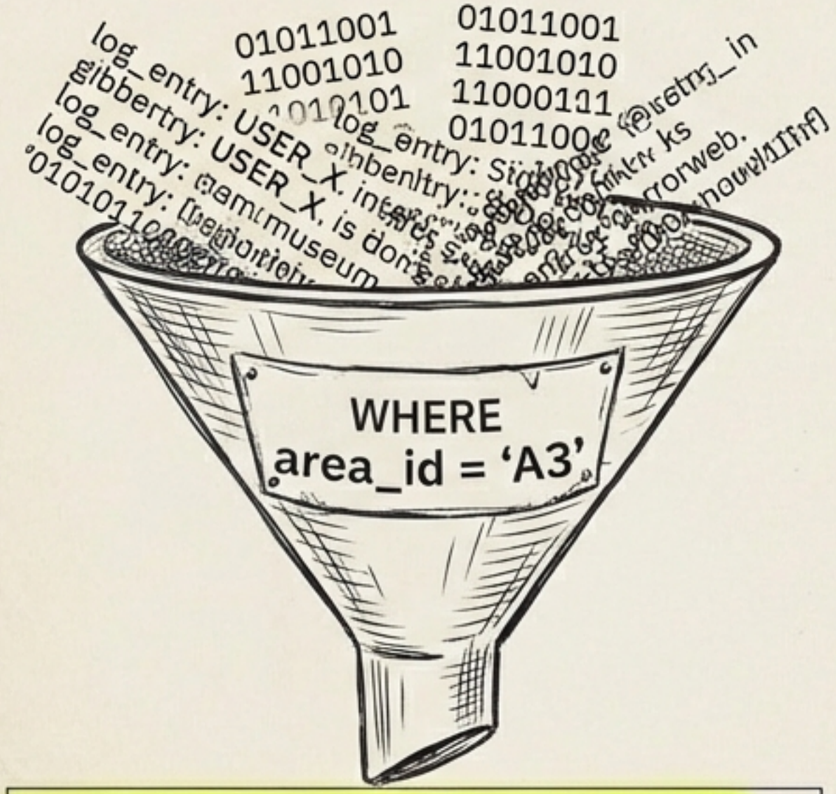
area_id	area_name	security_level
A1	Main Lobby	1
A2	Digital Vault	5
A3	Server Room	4
A4	Employee Lounge	2
A5	Curator's Office	3

Target Location Identified

The Server Room is Area ID A3. This is our primary filter for the security logs.

EVIDENCE

Phase 2: Auditing the Footprints



area_id: A3 | time: 22:14 | user: ADMIN_1
area_id: A3 | time: 22:30 | user: UNKNOWN
area_id: A3 | time: 23:05 | user: MAINTENANCE

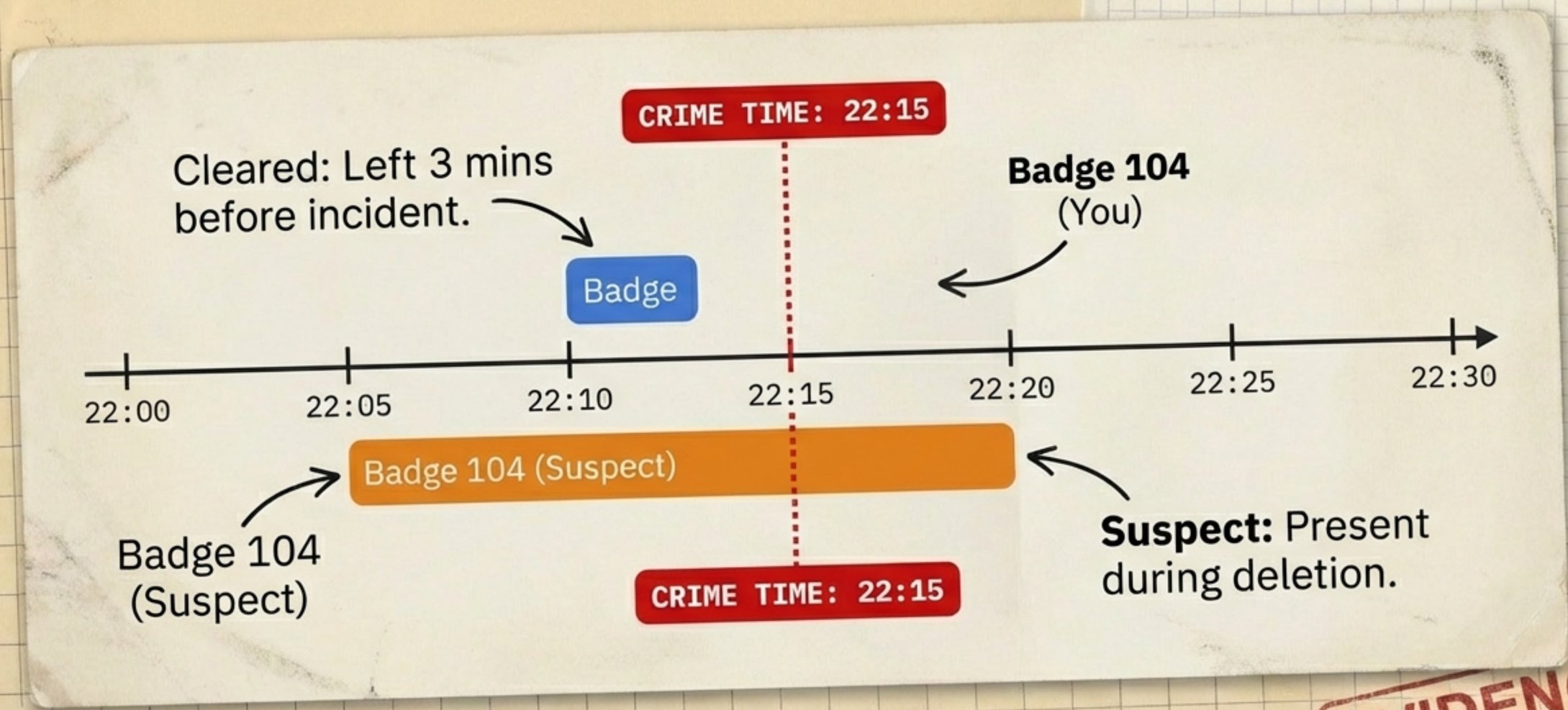
Terminal Window

```
> SELECT * FROM security_logs
  WHERE area_id = 'A3';
```

We can't look at every log entry in the museum. We use the WHERE clause as a magnifying glass to filter specifically for activity in the Server Room.

EVIDENCE

The Timeline Analysis



EVIDENCE

Phase 3: Unmasking the Identity


```
> SELECT name, role
  FROM employees
  WHERE badge_id = 104;
```

We have a number, but we need a name. We query the employees table to connect the WHERE clause to a badge ID person.

EVIDENCE

The Culprit Identified

The internal threat is confirmed. Elena Rodriguez, the IT Specialist, was the only one with access and opportunity. But just as we identified her, she launched a counter-attack.

EVIDENCE

The Sabotage

```
> UPDATE employees SET salary = 1;
```

Missing WHERE Clause!

badge_id	name	role	salary
101	Sarah Jenkins	Curator	1
102	Vicky Kumar	Admin	1
103	Marcus Thorne	Security	1
104	Elena Rodriguez	IT Specialist	1

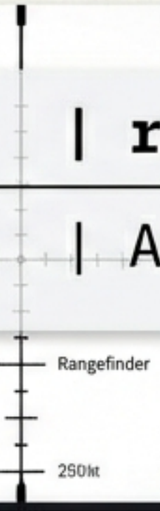
Elena executed a 'Scorched Earth' protocol. She ran an UPDATE without a filter, setting everyone’s salary to 1. The database is compromised.

EVIDENCE

Phase 4: The Surgical Fix

As the Database Admin (Badge 102), you must restore your salary to fund the recovery. Unlike the sabotage, we use a targeted command.

badge_id	name	role	salary
102	Vicky Kumar	Admin	82000



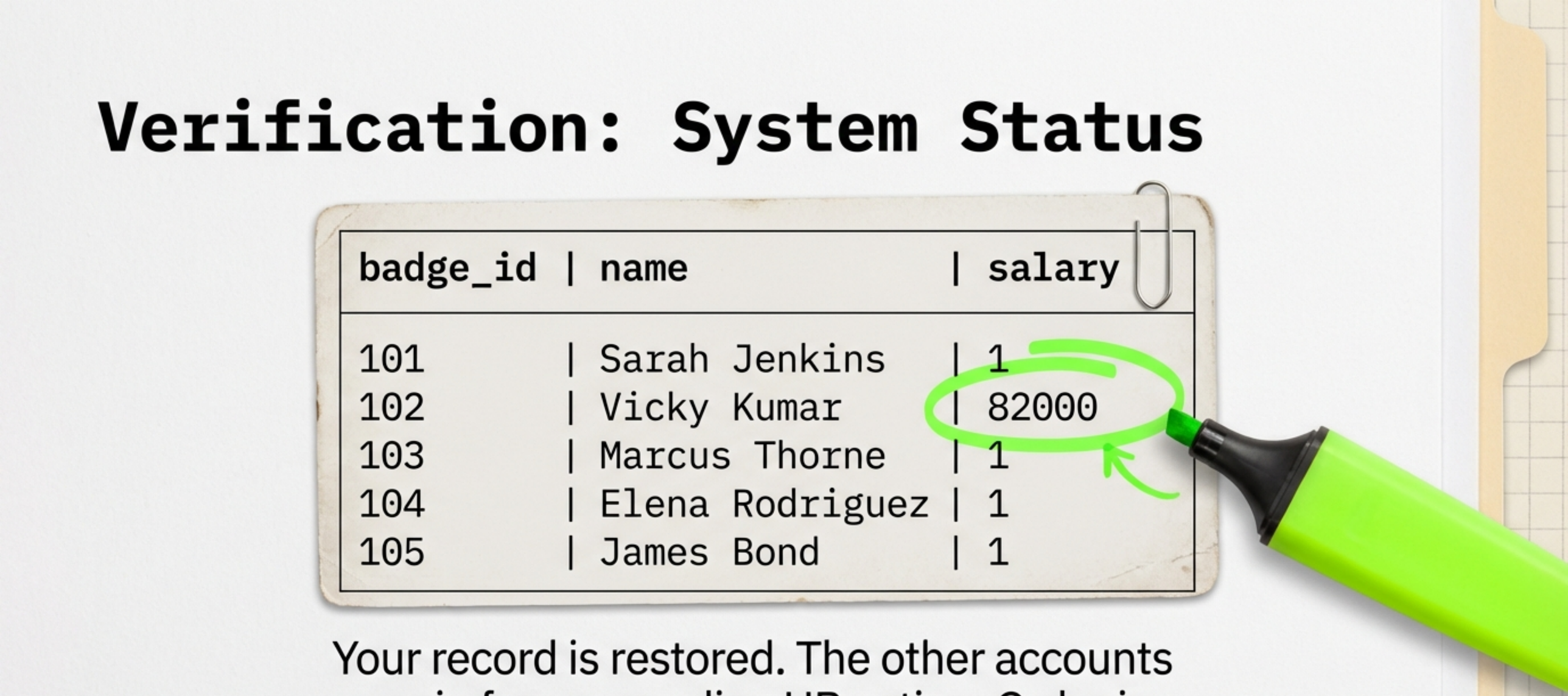

```
> UPDATE employees SET salary = 82000 WHERE badge_id = 102;
```

Safety Lock: The WHERE clause ensures only the target row is affected.

EVIDENCE

Verification: System Status

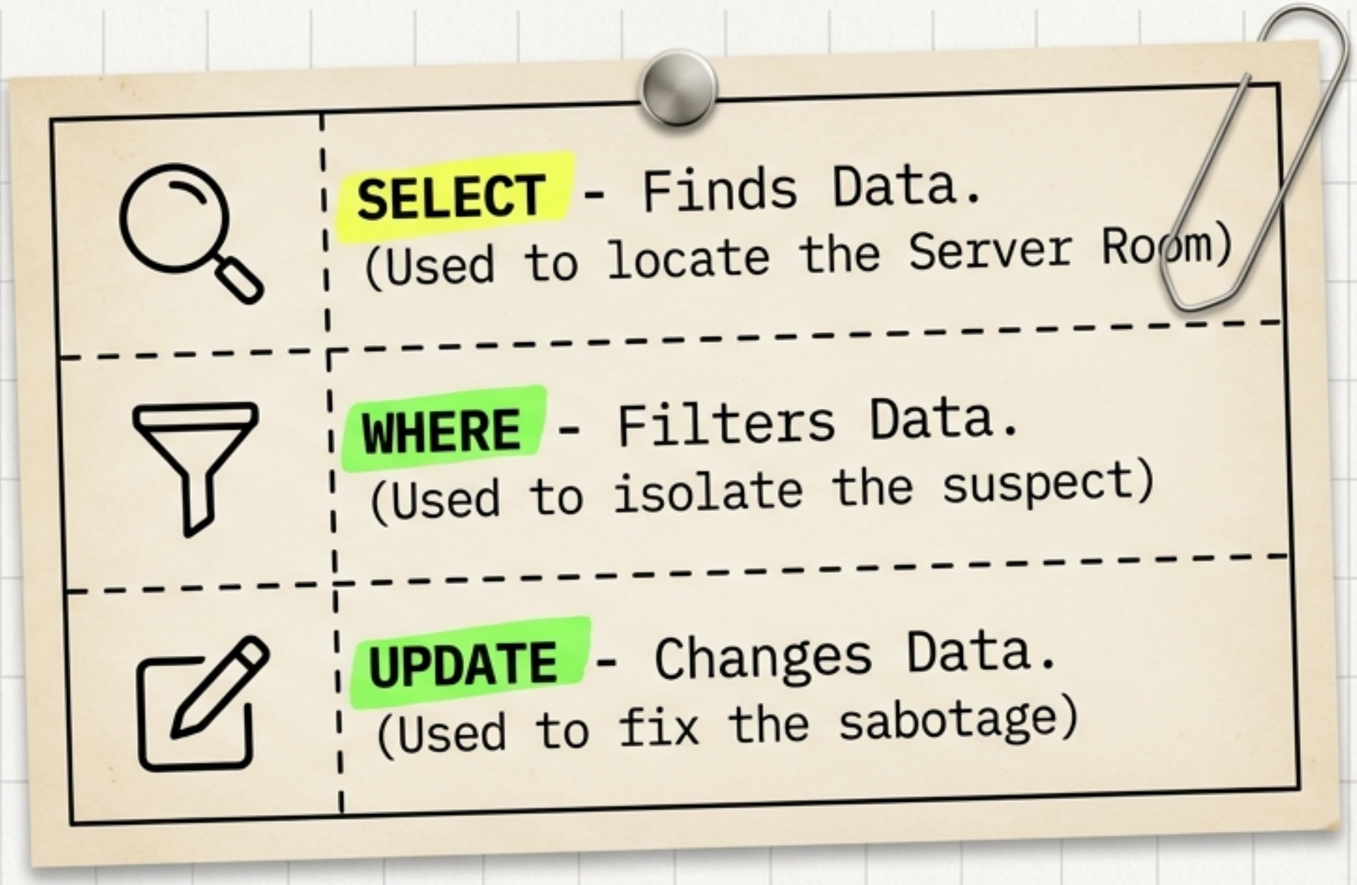
badge_id	name	salary
101	Sarah Jenkins	1
102	Vicky Kumar	82000
103	Marcus Thorne	1
104	Elena Rodriguez	1
105	James Bond	1



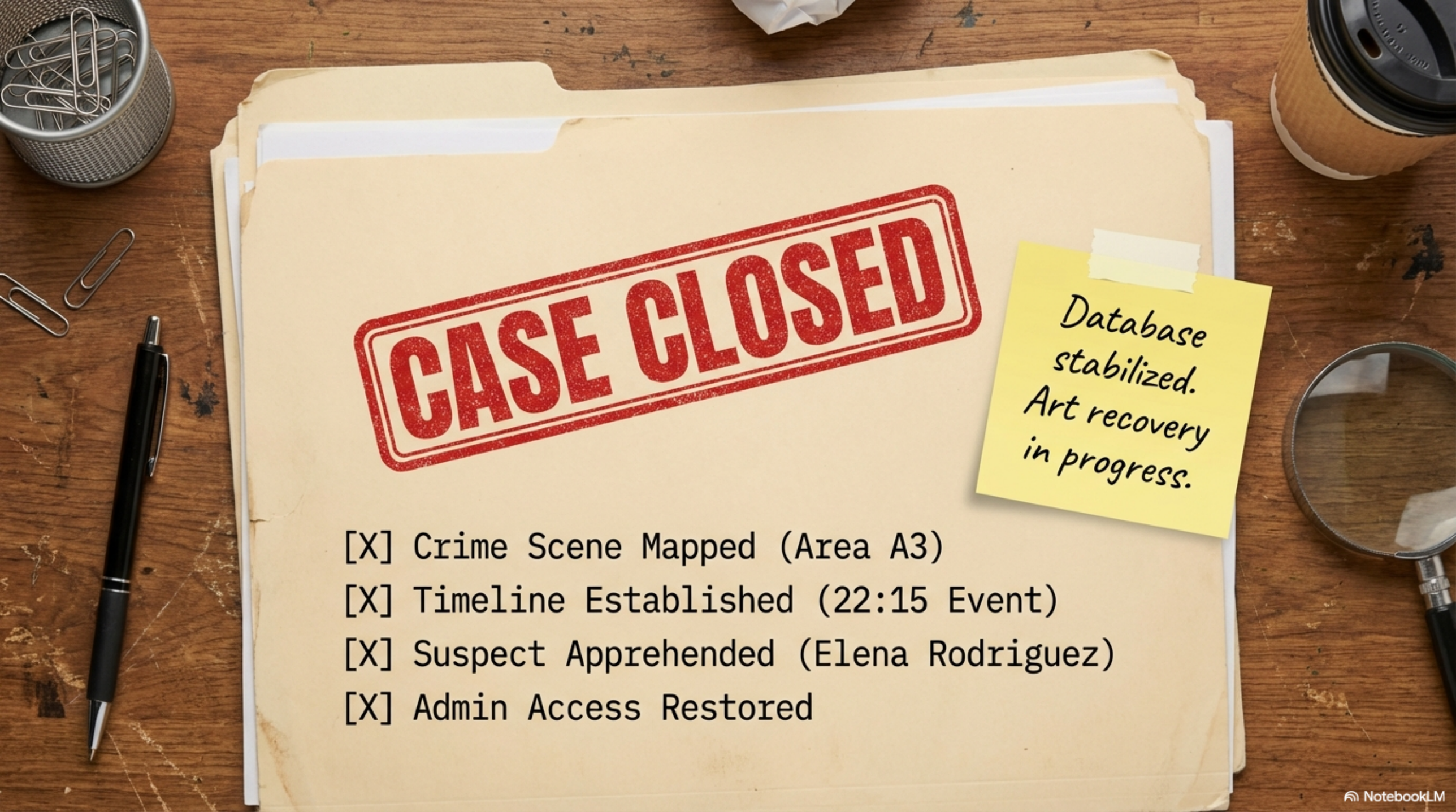
Your record is restored. The other accounts remain frozen pending HR action. Order is partially restored.

EVIDENCE

The Detective's Toolkit



"In SQL, a single missing keyword can cause chaos, but a precise command can restore order."


CASE CLOSED
Database stabilized. Art recovery in progress.
[X] Crime Scene Mapped (Area A3)
[X] Timeline Established (22:15 Event)
[X] Suspect Apprehended (Elena Rodriguez)
[X] Admin Access Restored